# ORF307_HW3_soln

## 1 ORF 307 HW3 Soln

## 2 Q1 Robust Approximate Solution of Least Squares

To solve this multi-objective problem, we stack our $A$ matrices and the $b_i$ vectors. We denote $\tilde{A}$ and $\tilde{b}$ to denote the stacked matrix and vector respectively.

$$\tilde{A} = \begin{bmatrix} A^{(1)} \\ \vdots \\ A^{(K)} \end{bmatrix}, \tilde{b} = \begin{bmatrix} b \\ \vdots \\ b \end{bmatrix}$$

The problem statement says to assume that $A$ has linearly independent columns. Thus, $\tilde{A}$ also has linearly independent columns. So this is a standard least squares problem now, and we can apply the normal equations $\tilde{A}^T\tilde{A}x^{rob} = \tilde{A}^T\tilde{b}$. Since $\tilde{A}$ is in this block form, we can further simplify.

$$\tilde{A}^T\tilde{A} = \sum_{i=1}^{K} (A^{(i)})^T A^{(i)}$$

$$\tilde{A}^T\tilde{b} = (\sum_{i=1}^{K} A^{(i)})^T b$$

Thus, our solution simplfies to

$$x^{rob} = (\sum_{i=1}^{K} (A^{(i)})^T A^{(i)})^{-1} (\sum_{i=1}^{K} A^{(i)})^T b$$

We see that if $K = 1$, this simplifies to

$$x^{rob} = ((A^{(1)})^T A^{(1)})^{-1} (A^{(1)})^T b = (A^{(1)})^{-1} ((A^{(1)})^T)^{-1} (A^{(1)})^T b = (A^{(1)})^{-1} b,$$

our solution to the standard least squares problem with only one objective term.

## 3 Q2 Least Norm Polynomial Interpolation

(a) We look at each of the 4 interpolation conditions individually and express them as conditions on the coefficients

$$p(0) = 0 \implies c_1 = 0$$

$$p'(0) = 0 \implies c_2 = 0$$
$$p(1) = 1 \implies c_1 + c_2 + c_3 + c_4 + c_5 = 1$$
$$p'(1) = 0 \implies c_2 + 2c_3 + 3c_4 + 4c_5 = 0$$

Now writing this in matrix form yields

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

This system of equations is underdetermined since there are 5 variables and only 4 equations. Thus, there are infinitely many different coefficients that satisfy these constraints.

(b) We need to solve the following constrained optimization problem:

$$\begin{array}{ll} \text{minimize} & \|c\|^2 \\ \text{subject to} & Ac - b = 0 \end{array} \tag{1}$$

Note that the objective, $\|c\|^2$, can be written in a least squares form:

$$\|c\|^2 = c^T c = c^T I^T I c - 0 = \|Ic - 0\|^2$$

where $I$ is the identity matrix, and 0 is the zero vector (all of appropriate dimensions). We can then write the original optiization problem as the constrained least squares problem

$$\begin{array}{ll} \text{minimize} & \|Ic - 0\|^2 \\ \text{subject to} & Ac - b = 0 \end{array} \tag{2}$$

The optimal solution of the above problem will give us the coefficients of the 4th degree polynomial we look for.

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import lu


################################
# Let's first define our data
################################

I = np.eye(5) # the identity matrix
A = np.array([[1, 0, 0, 0, 0],
              [0, 1, 0, 0, 0],
              [1, 1, 1, 1, 1],
              [0, 1, 2, 3, 4]])
z = np.array([0, 0, 0, 0, 0]) # the zero vec
b = np.array([0, 0, 1, 0])
```

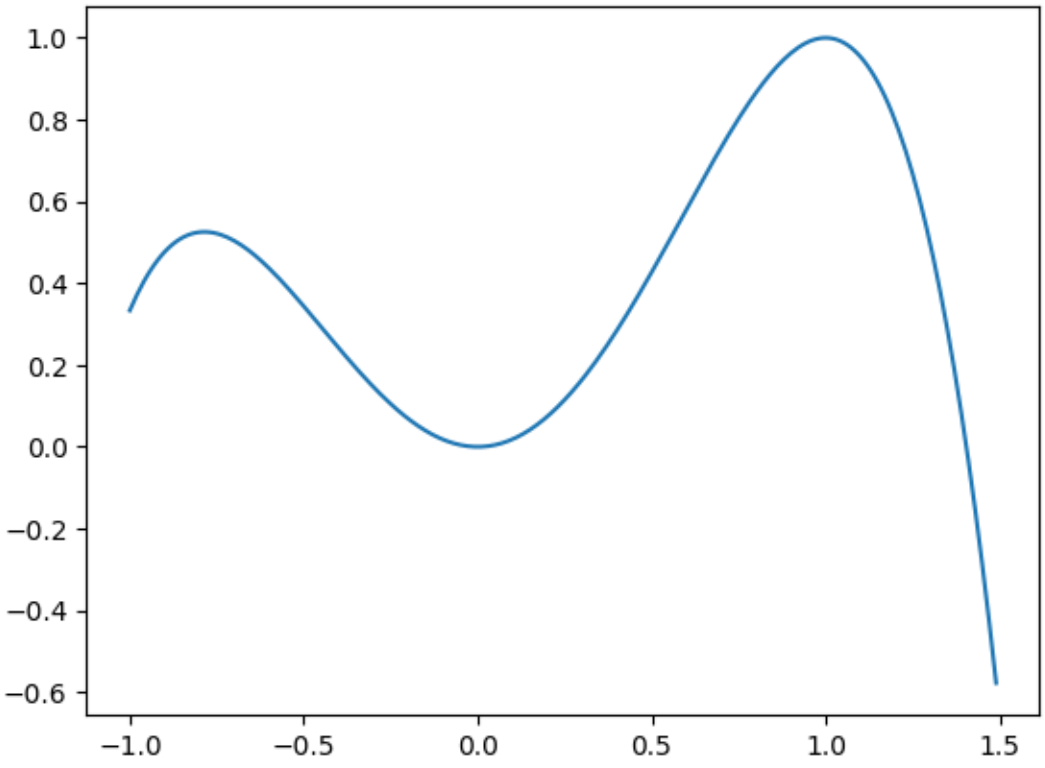
```
##########################################################
# Next let's define the functions we are going to use
##########################################################

def KKT_matrix_rhs(A, b, C, d):
    """
    returns the KKT matrix and the right hand side vector that describes the
    linear system to solve the constrained least squares problem
    """
    m, n = A.shape
    p, n = C.shape

    G = A.T @ A  # Gram matrix
    KKT = np.block([[2*G, C.T],
                    [C, np.zeros((p, p))]])
    rhs = np.hstack([2*A.T @ b, d])
    return KKT, rhs

def forward_substitution(L, b):
    n = L.shape[0]
    x = np.zeros(n)
    for i in range(n):
        x[i] = (b[i] - L[i,:i] @ x[:i])/L[i, i]
    return x

def backward_substitution(U, b):
    n = U.shape[0]
    x = np.zeros(n)
    for i in reversed(range(n)):
        x[i] = (b[i] - U[i,i+1:] @ x[i+1:])/U[i, i]
    return x

def solve_via_lu(P, L, U, b):
    '''
    Solve linear system Ax = b where
        A = PLU
    '''
    x1 = P.T @ b
    # now LUx = x1
    x2 = forward_substitution(L, x1)
    # now Ux = x2
    x = backward_substitution(U, x2)
    return x


##########################################################
```

```
# Now we can write the main code that solves the
# constrained least square optimization problem
##################################################


# first form the KKT system of equations
KKT, rhs = KKT_matrix_rhs(I, z, A, b)

# now, solve the linear system: KKT @ x = rhs
P, L, U = lu(KKT)
x = solve_via_lu(P, L, U, rhs)
coeffs = x[:5]
print('coefficients', coeffs)
```

```
coefficients [ 0.          0.          1.83333333  0.33333333 -1.16666667]
```

[2]:
```
# now plot the polynomial
xx = np.arange(-1, 1.5, .01)
yy = coeffs[0] + coeffs[1]*xx + coeffs[2]*xx**2 + coeffs[3]*xx**3 +␣
 ↪coeffs[4]*xx**4
plt.plot(xx, yy)
```

[2]:
```
[<matplotlib.lines.Line2D at 0x1c06fd6ea10>]
```



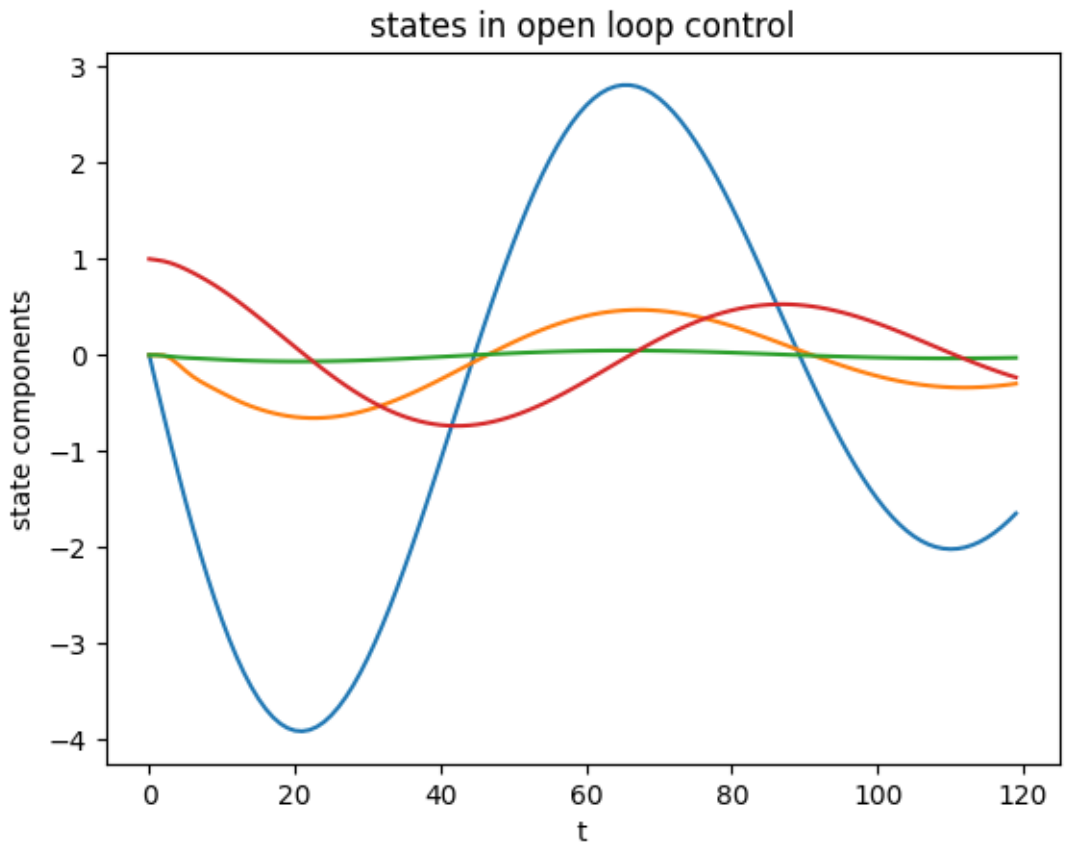
We see that $p(0) = 0$ and $p(1) = 1$. We also see that the slope is flat at zero and one, enforcing the two derivative constraints.

# 4 Q3 Linear Quadratic Control

(a)

[3]:
```
import numpy as np
import matplotlib.pyplot as plt
A = np.array([[.99, .03, -.02, -.32], [.01, .47, 4.7, 0], [.02, -.06, .4, 0], [.
↪01, -.04, .72, .99]])
B = np.array([[.01, .99], [-3.44, 1.66], [-.83, .44], [-.47, .25]])
n, m = B.shape
C = np.eye(n)
x_init = np.array([0, 0, 0, 1])
x_des = np.zeros(n)
T = 120
rho = 100

x = np.zeros((T, n))
x[0,:] = x_init

for i in range(1, T):
    x[i,:] = A @ x[i-1,:]
plt.plot(x)
plt.title('states in open loop control')
plt.xlabel('t')
plt.ylabel('state components')
```

[3]:
```
Text(0, 0.5, 'state components')
```




(b)

[4]:
```
def KKT_matrix_rhs(A, b, C, d):
    """
    returns the KKT matrix and the right hand side vector that describes the
    linear system to solve the constrained least squares problem
    """
    m, n = A.shape
    p, n = C.shape

    G = A.T @ A  # Gram matrix
    KKT = np.block([[2*G, C.T],
                    [C, np.zeros((p, p))]])
    rhs = np.hstack([2*A.T @ b, d])
    return KKT, rhs
```

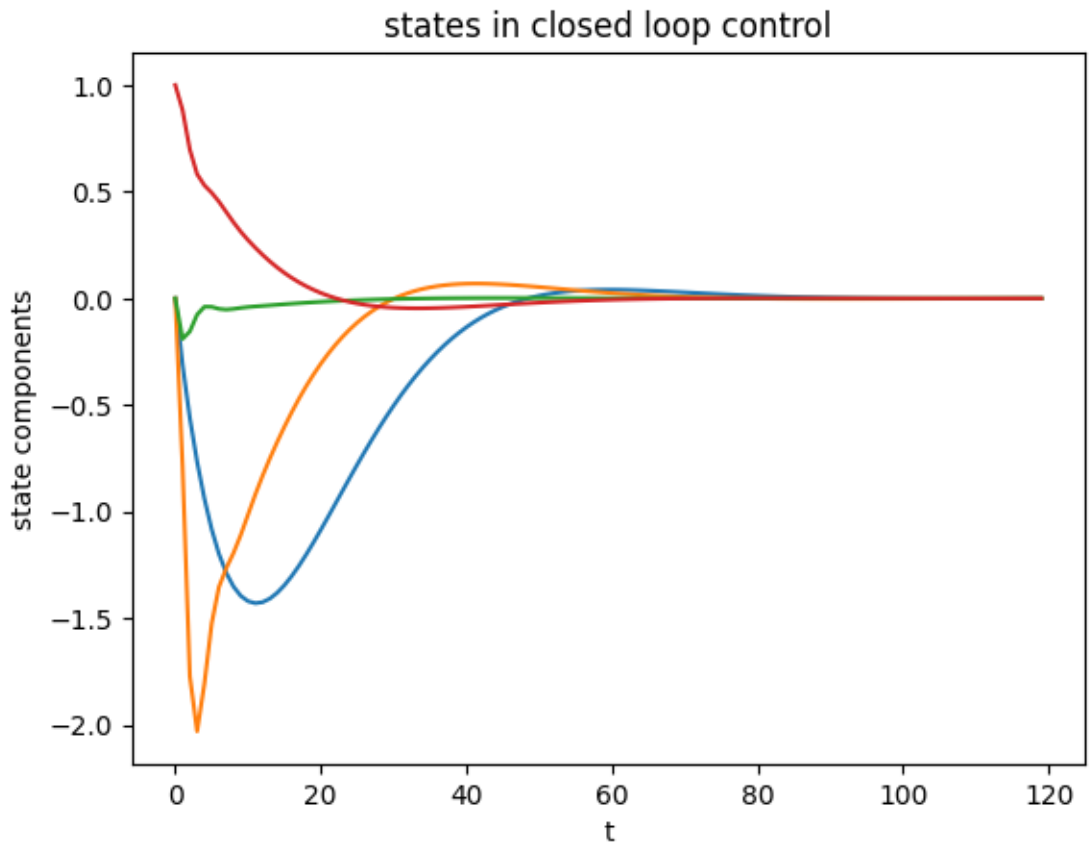
[5]:
```
def construct_tilde_matrices(A, B, C, x_init, x_des, T, rho):
    """
    Construct A_tilde, b_tilde, c_tilde, d_tilde.
    """
```

```
    n, m = B.shape
    p = C.shape[0]

    A_tilde = np.block([[np.kron(np.eye(T), C), np.zeros((p * T, m * (T-1)))],
                        [np.zeros((m * (T - 1), n * T)), np.sqrt(rho) * np.
 ↪eye(m * (T-1))]])

    # construct btilde
    b_tilde = np.zeros(p * T + m * (T - 1))

    # construct Ctilde bit by bit
    C_tilde11 = np.hstack([np.kron(np.eye(T-1),A), np.zeros((n*(T-1),n))]) \
        - np.hstack([np.zeros((n*(T-1),n)), np.eye(n*(T-1))])
    C_tilde12 = np.kron(np.eye(T-1), B)
    C_tilde21 = np.block([[np.eye(n), np.zeros((n,n*(T-1)))],
                          [np.zeros((n,n*(T-1))), np.eye(n)]])
    C_tilde22 = np.zeros((2 * n, m * (T-1)))

    C_tilde = np.block([[C_tilde11, C_tilde12],
                        [C_tilde21, C_tilde22]])

    # construct dtilde
    d_tilde = np.hstack([np.zeros(n * (T - 1)), x_init, x_des])

    return A_tilde, b_tilde, C_tilde, d_tilde
```

[6]:
```
Atil, btil, Ctil, dtil = construct_tilde_matrices(A, B, C, x_init, x_des, T,␣
 ↪rho)

KKT, rhs = KKT_matrix_rhs(Atil, btil, Ctil, dtil)

# reuse our solve_via_lu function
P, L, U = lu(KKT)
z = solve_via_lu(P, L, U, rhs)

x = [z[i*n:(i+1)*n] for i in range(T)]
u = [z[n*T+i*m: n*T+(i+1)*m] for i in range(T-1)]
y = [C @ xt for xt in x]
```

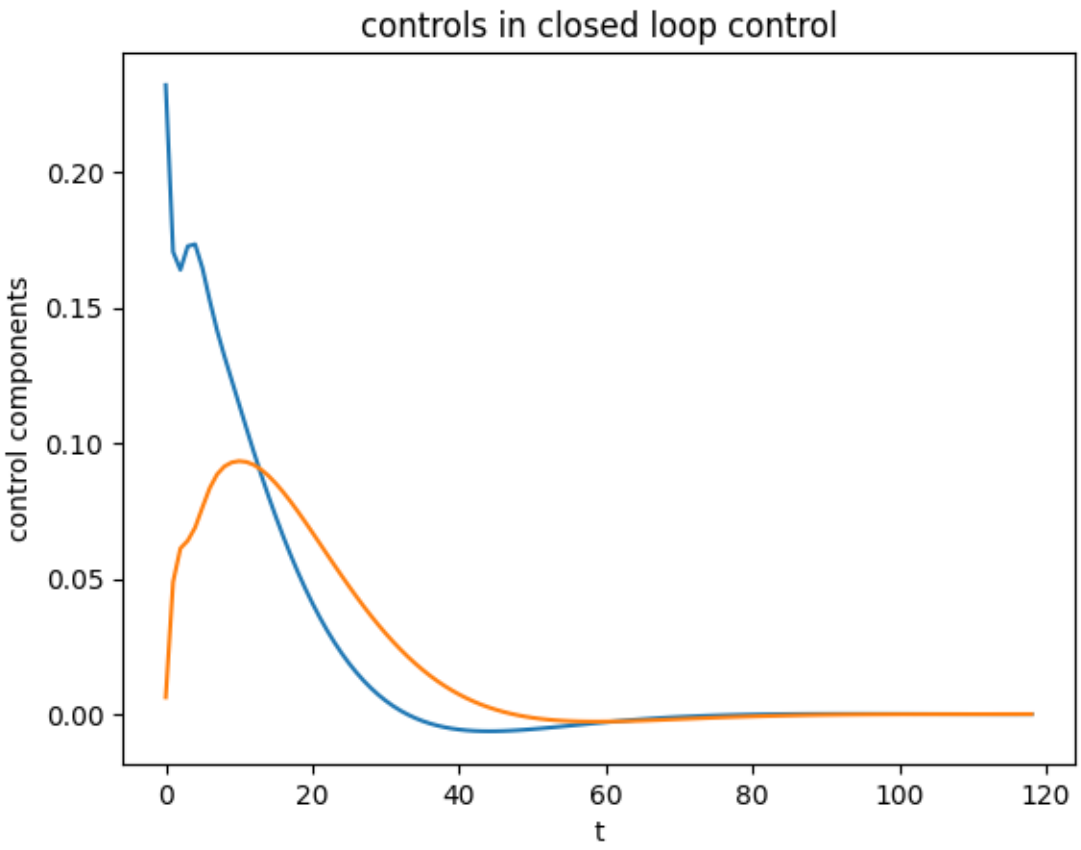
[7]:
```
plt.plot(x)
plt.title('states in closed loop control')
plt.xlabel('t')
plt.ylabel('state components')
```

[7]:
```
Text(0, 0.5, 'state components')
```




[8]:
```
plt.plot(u)
plt.title('controls in closed loop control')
plt.xlabel('t')
plt.ylabel('control components')
```

[8]: Text(0, 0.5, 'control components')


controls in closed loop control
control components
0.20
0.15
0.10
0.05
0.00
0
20
40
60
80
100
120
t

[ ]: